
OCNR: Stabilizing Self-Play by Mitigating Iteration-Collapse With One-Class Novelty Rewards

Seungyoo Lee¹ Giung Nam¹ Hyungi Lee^{2†} Juho Lee^{1†}

Abstract

Training large language models via self-play often suffers from a persistent iteration-collapse, where performance initially improves but subsequently regresses as training iterations increase. We analyze this phenomenon as arising from cross-iteration degeneration, where the task-generation distribution becomes increasingly confined to a narrow subset of familiar (seen) problems, weakening the effective learning signal and destabilizing training. To address this issue, we propose a plug-in approach that augments existing self-play pipelines with a one-class novelty reward. A Seen Detector trained on a historical buffer of previously used training problems identifies in-support instances and discourages redundant generation by the questioner, thereby steering exploration toward under-explored yet learnable regions. Experimental results show that the proposed method mitigates iteration-collapse during iterative training and yields consistent improvements.

1. Introduction

Recent advances in post-training demonstrate that reinforcement learning (RL) can substantially improve the reasoning abilities of large language models when reliable training signals are available (Lambert et al., 2025; Guo et al., 2025). In particular, recent successes with RL using verifiable rewards and critic-free policy optimization methods, such as Group Relative Policy Optimization (GRPO; Shao et al., 2024), highlight the central role of well-structured reward signals in effective post-training. Motivated by these results, a natural question is whether such improvements can be sustained without access to large-scale, human-curated

supervision. This question has led to growing interest in *self-play* and *self-evolving* training loops, in which models alternate between generating tasks and solving them, relying on automatically obtained signals for learning, e.g., self-consistency (Chen et al., 2025; Huang et al., 2025) or programmatic verification (Zhao et al., 2025). In principle, these closed-loop procedures offer a path toward open-ended improvement, allowing models to continually refine their reasoning capabilities as they become more capable.

In practice, however, iterative self-play often fails to scale with additional training iterations. A recurring failure mode is *iteration-collapse*, where performance initially improves but subsequently regresses as training continues (Huang et al., 2025). This behavior is qualitatively different from saturation or plateauing, where gains merely slow down. Instead, iteration-collapse corresponds to post-peak degradation, meaning that further self-play can actively harm generalization on held-out benchmarks. Such instability limits the usefulness of long-run self-evolution, complicates training schedules, and weakens the reliability of self-play as a general recipe for continual improvement.

In this work, we first analyze iteration-collapse as a consequence of cross-iteration degeneration in task generation. As self-play iterations proceed, we hypothesize that the task-generation distribution increasingly drifts toward previously learned regions, which we refer to as the *seen region*. Repeated regeneration of in-support tasks diminishes the effective learning signal and induces feedback loops that amplify existing biases. This phenomenon aligns with broader observations that self-generated training signals can become misaligned or exploitable, allowing optimization to succeed under proxies that fail to track the true objective (Shao et al., 2025). The key challenge is therefore not merely to generate tasks, but to sustain meaningful exploration across iterations without drifting into redundant or degenerate regimes.

Existing approaches address aspects of this challenge but leave a gap for minimal, closed-loop self-play settings. Some methods improve stability by introducing external grounding or environments. For example, SPICE (Liu et al., 2025) leverages a large document corpus to provide grounded rewards and sustained external signal, while SPELL (Yang et al., 2025b) generates document-based ques-

¹Kim Jaechul Graduate School of AI, KAIST, Seoul, South Korea ²Department of AI, Kookmin University, Seoul, South Korea. Correspondence to: Hyungi Lee <lhk2708@kookmin.ac.kr>, Juho Lee <juholee@kaist.ac.kr>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

tions with verifier-guided curricula to support long-context reasoning. Although effective, such approaches rely on additional infrastructure and assumptions beyond self-play from zero external data. Another line of work explores label-free or self-reward signals that avoid reliance on ground-truth supervision. Entropy-based and confidence-based objectives can strengthen reasoning by reinforcing high-confidence trajectories (Agarwal et al., 2025; Prabhudesai et al., 2025), and Co-Reward (Zhang et al., 2025) introduces cross-view self-supervision to mitigate collapse via contrastive agreement. Together, these approaches underscore that stability in self-play critically depends on the structure of the reward signal and on avoiding degenerate shortcuts, yet they do not directly address cross-iteration degeneration in minimal self-play settings.

Motivated by these observations, we propose a novel plug-in stabilization mechanism for self-play, which we refer to as **One-Class Novelty Reward (OCNR)**. At a high level, OCNR introduces a novel reward model that evaluates whether tasks generated by the questioner fall within previously learned, redundant regions of the task space. By discouraging the generation of in-support instances, the proposed reward promotes exploration toward under-explored yet learnable tasks, directly targeting iteration-collapse. Crucially, OCNR operates in a fully closed-loop manner and preserves the underlying self-play pipeline. The reward model is continuously updated across iterations to reflect the evolving set of generated tasks, ensuring that redundancy is assessed relative to the current training history. The resulting novelty-aware reward signal is then used within standard RL-based self-play training. With this design in place, we empirically demonstrate that OCNR mitigates post-peak performance regression in iterative self-play and yields consistent improvements over baseline methods under compute-matched settings.

Contributions are as follows:

- We analyze iteration-collapse in closed-loop self-play and identify cross-iteration degeneration of the task-generation distribution as a primary source of post-peak performance regression.
- We propose a simple, plug-in one-class novelty reward that stabilizes self-play by discouraging redundant in-support task generation while preserving a fully closed-loop training pipeline.
- We empirically demonstrate consistent improvements under compute-matched iterative training, supported by ablations and distributional analyses.

2. Preliminaries

2.1. Problem setup

We consider a problem-solving setting in which questions q are drawn from an underlying distribution $\mathcal{D}$, and a *solver*

model π_S aims to produce correct solutions for such questions. Since π_S is imperfect, our objective is to improve its problem-solving performance through iterative training. In standard supervised settings, such improvement typically relies on large collections of human-annotated problem solution pairs, which are costly to obtain and difficult to scale. To avoid this dependence on human supervision, we adopt a *self-play* paradigm (Samuel, 1959); within the LLM context, this typically involves a *questioner* (also referred to as a *proposer* or *challenger*) model π_Q designed to generate training questions for the solver (Zhao et al., 2025; Chen et al., 2025; Huang et al., 2025). At each iteration, π_Q produces candidate questions that are solved by π_S , and the solver’s responses derive training signals, allowing both models to be trained iteratively to maximize the performance of π_S over $\mathcal{D}$. However, because π_Q is trained using feedback from the current π_S , its generation strategy can become increasingly coupled to the solver’s evolving behavior, leading to reduced diversity, redundant or ambiguous questions, and ultimately the iteration-collapse failure discussed earlier.

2.2. Self-evolving LLMs via self-play

Concretely, at each iteration t , the questioner generates a question $q_t \sim \pi_Q$, and the solver produces a corresponding answer $a_t \sim \pi_S(\cdot|q_t)$. A verifier or self-consistency mechanism then evaluates the output to yield a scalar reward R_S and it is typically defined as a simple binary reward reflecting correctness relative to a reference a_t^* , which can be either the ground-truth answer or a majority-voted pseudo-answer (Zuo et al., 2025; Shafayat et al., 2025):

$$R_S(q_t, a_t) = \mathbb{I}[a_t = a_t^*]. \quad (1)$$

While Equation (1) improves the performance of π_S , the framework simultaneously incentivizes π_Q to generate questions that are both learnable and informative, creating a self-reinforcing dynamic curriculum. To this end, a questioner reward R_Q should be designed to capture three complementary properties: (i) *validity*: whether the generated question is valid, (ii) *difficulty*: whether its difficulty is appropriate for the current solver, and (iii) *diversity*: whether it is diverse relative to other generated problems. Regarding (i) *validity*, it can be readily enforced through a format-aware penalty (Guo et al., 2025), where rewards are strictly contingent upon the output adhering to the prescribed format. On the other hand, there is no established gold standard for addressing (ii) *difficulty* and (iii) *diversity*; thus, identifying effective reward mechanisms that adequately address both properties remains an active area of investigation. Recent methodologies approach these challenges as follows:

- Absolute Zero (Zhao et al., 2025) ensures (ii) *difficulty* through the following reward function:

$$R_Q^{AZ}(q_t) = (1 - p_{\text{solved}}(q_t))\mathbb{I}[p_{\text{solved}}(q_t) \neq 0], \quad (2)$$

where $p_{\text{solved}}(q_t)$ is the success rate of π_S on question q_t , estimated by averaging the solver reward R_S across K stochastic roll-outs at a non-zero temperature, i.e.,

$$p_{\text{solved}}(q_t) = \frac{1}{K} \sum_{k=1}^K R_S(q_t, a_t^{(k)}), \quad (3)$$

for $a_t^{(1)}, \dots, a_t^{(K)} \sim \pi_S(\cdot | q_t)$ with the ground-truth reference a_t^* . It effectively filters out both trivial ($\hat{p} = 1$) and unsolvable ($\hat{p} = 0$) problems to maintain a productive learning signal. However, (iii) *diversity* is merely managed at the prompting level, rather than an explicit reward, by instructing the model to generate questions that differ from those in previous iterations.

- R-Zero (Huang et al., 2025) ensures (ii) *difficulty* via the following symmetric reward function:

$$R_Q^{\text{RZ,unc}}(q_t) = (1 - 2|\hat{p}_{\text{solved}}(q_t) - 1/2|), \quad (4)$$

where $\hat{p}_{\text{solved}}$ is computed identically to p_{solved} , but utilizes the majority-voted pseudo-answer as the reference a_t^* for evaluation instead of the ground-truth. It effectively incentivizes questions where π_S is maximally uncertain and yields the strongest gradient signals for learning (Sun et al., 2025). Furthermore, R-Zero explicitly handles (iii) *diversity* by incorporating a repetition penalty directly into the reward objective, penalizing questions that are structurally similar to those concurrently generated in the iteration¹:

$$R_Q^{\text{RZ,rep}}(q_t) = -\frac{\{\# \text{ questions similar to } q_t\}}{\{\# \text{ questions generated}\}} \quad (5)$$

The final questioner reward for R-Zero is then defined as the sum of the uncertainty and repetition terms:

$$R_Q^{\text{RZ}}(q) = R_Q^{\text{RZ,unc}}(q) + R_Q^{\text{RZ,rep}}(q). \quad (6)$$

3. OCNR: One-Class Novelty Reward

3.1. Overview

Regarding the (iii) *diversity* component, the prevailing consensus in the literature focuses on preventing the solver from repeatedly learning the same problems. Existing strategies, such as prompt engineering (Zhao et al., 2025) and repetition-aware reward mechanisms (Huang et al., 2025), prioritize maximizing problem variety to avoid such redundancy. However, we argue that such diversity is essentially

¹Under our notation, R-Zero produces a batch of questions for each iteration. Beyond the distinctions in prompting (Absolute Zero) and explicit reward mechanisms (R-Zero), a key difference lies in the reference for diversity: Absolute Zero evaluates each new question against the cumulative history of all prior iterations, whereas R-Zero assesses it exclusively relative to concurrent questions within the same batch.

a *proxy for novelty*; for self-play to be effective, the questioner must generate problems that are not only diverse but specifically *novel* relative to the solver’s current state.

Accordingly, we propose the **One-Class Novelty Reward (OCNR)**, a history-aware reward designed to encourage the questioner π_Q to generate problems that have not been previously utilized to train the solver π_S . OCNR provides π_Q with an explicit signal indicating whether a generated problem lies within the solver’s seen training support, and can be viewed as a direct form of novelty reward that operates by incentivizing the questioner to target the solver’s current blind spots, rather than simply encouraging superficial diversity. To this end, we first design a lightweight Seen Detector (SD) that determines whether a generated problem is in-support (Section 3.2), and then describe how the SD output is converted into the OCNR signal (Section 3.3). Finally, we provide details on the joint training of the SD together with π_S and π_Q (Section 3.4).

3.2. Seen detector

We begin by defining the *seen detection* task, the foundational operation of the Seen Detector (SD) module within our framework. Unlike conventional binary classification, which imposes a discrete boundary between *seen* and *unseen* classes, our approach focuses on quantifying the degree of *seen-ness*. Our formulation is closely aligned with the notion of *one-class classification* (Moya & Hush, 1996), reflecting the observation that while positive (*seen*) examples are typically accessible, the “number and variety of negatives (*unseen*) is not well modeled” (Scheirer et al., 2013). Given this inherent difficulty in modeling the unseen space, we adopt a detection strategy centered on *seen-ness* rather than explicit binary discrimination. Recognizing that detection can be further refined by incorporating available unseen samples into the training process (Wu & Ye, 2009), we leverage both seen and unseen instances to effectively tighten the decision boundary surrounding the seen data.

Specifically, for a given question $q_t \sim \pi_Q$, let $\mathbf{h}(q_t) \in \mathbb{R}^d$ denote its embedding extracted from the solver π_S , and let $\mathbf{w}_s \in \mathbb{R}^d$ denote the prototype maintained by the SD, which summarizes the solver’s seen training support. The degree of *seen-ness* is then quantified as:

$$\text{SEEN}(q_t) = \exp\left(-\frac{\|\mathbf{h}(q_t) - \mathbf{w}_s\|_2^2}{\tau}\right), \quad (7)$$

where τ is a temperature parameter controlling the sensitivity of the detection. The choice to utilize the solver’s output embedding space reflects a crucial requirement: in our self-play paradigm, *seen detection* must be anchored to the perspective of the solver π_S . By operating directly on the feature space of π_S , the SD ensures that *seen-ness* is evaluated within the specific manifold that directly governs

the solver’s learning process. While the precise details are provided in Section 3.4, the prototype w_s is optimized by minimizing a binary cross-entropy (BCE) loss, with seen and unseen instances labeled as 1 and 0, respectively.

At each iteration, the SD provides the supervision signal required to update π_Q so as to discourage the generation of previously exploited training instances. Concretely, the SD maintains a compact summary of the solver’s seen training support in the form of the learnable prototype vector w_s in the solver feature space. This prototype serves as a memory of problems that have already contributed to solver updates across previous iterations. Instead of fitting a highly expressive classifier with a rapidly shifting decision boundary, SD evaluates newly generated problems by comparing their solver embeddings to this accumulated support representation. Such a memory-based formulation intentionally emphasizes stability over adaptivity, allowing SD to capture cross-iteration redundancy in a consistent manner while remaining lightweight and computationally efficient. As we show later, this design choice is sufficient to provide a reliable novelty signal in our self-play pipeline and avoids overfitting to transient generation patterns of π_Q .

3.3. Novelty reward

Following the vector-based memory formulation of the SD introduced in Section 3.2, we now describe the proposed novelty reward, OCNR, which provides an explicit learning signal to train the questioner policy π_Q to avoid generating problems that have already been exploited during solver training. Unlike prior works that rely on generic, language-level diversity (Zhao et al., 2025; Huang et al., 2025), our approach yields a learning signal for π_Q that is specifically tailored to the evolving state of π_S at each iteration.

Distance-based OCNR. We propose a distance-based novelty reward that directly measures the proximity of a generated problem to the solver’s seen training support using Equation (7). Specifically, the OCNR is defined as:

$$R_Q^{\text{nov}}(q_t) = 1 - \text{SEEN}(q_t), \quad (8)$$

which assigns a low reward only to problems that lie close to the seen support prototype, while allowing problems farther away in the embedding space to retain high novelty.

Sigmoid-based OCNR. Alternatively, by viewing the *seen detection* task as a conventional binary classification problem, one could formulate a sigmoid-based OCNR using the inner product between the prototype and the embedding:

$$R_Q^{\text{nov-sig}}(q_t) = 1/(1 + \exp\{w_s^\top h(q_t)\}). \quad (9)$$

However, we argue that this formulation is less suitable for our framework. As illustrated in the left panel of Figure 1, such a reward induces a global half-space decision boundary orthogonal to w_s . Consequently, problems lying on the

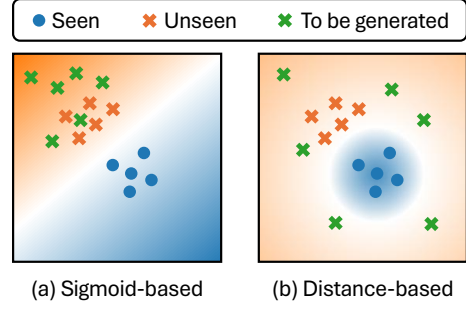


Figure 1. **Comparing novelty rewards.** Conceptual illustration of novelty rewards: (a) sigmoid-based reward induces a global half-space boundary, while (b) distance-based reward establishes a localized boundary surrounding the seen instances.

seen side of this hyperplane are uniformly penalized, regardless of their actual position relative to the solver’s training support. As a result, the sigmoid-based formulation limits exploration across an entire region, even if vast portions of that region remain unexploited by the solver (cf. the lower and right areas in the left panel of Figure 1).

In contrast, the distance-based formulation provides a soft, locality-sensitive notion of redundancy that penalizes only problems that are genuinely close to previously exploited *seen* questions, as shown in the right panel of Figure 1. This localized behavior aligns more closely with the objective of OCNR: discouraging redundant reuse of the solver’s training support while preserving flexibility for π_Q to explore diverse and under-explored regions of the problem space. Accordingly, we adopt the distance-based novelty reward as the default for all subsequent experiments.

3.4. Training procedure

To detail our training procedure, we primarily describe its integration within the R-Zero framework (Huang et al., 2025). However, it is important to note that our approach generalizes to other self-play paradigms; this versatility stems from the modular, plug-and-play nature of OCNR. Integration into the self-play loop is achieved by simply incorporating the SD module and augmenting the questioner’s reward signal with the OCNR. Consequently, the proposed method can be seamlessly adopted by any self-play pipeline that employs a questioner-solver architecture similar to R-Zero. Overall, our training follows an alternating self-play loop consisting of the following stages:

Stage 0. Warm-start without OCNR. At the first iteration ($t = 1$), SD has not yet accumulated a meaningful summary of the solver’s seen support. We therefore run the underlying self-play procedure without OCNR to obtain an initial solver update. After this update, we initialize SD from the updated solver representations and activate OCNR from the next iteration ($t > 1$).

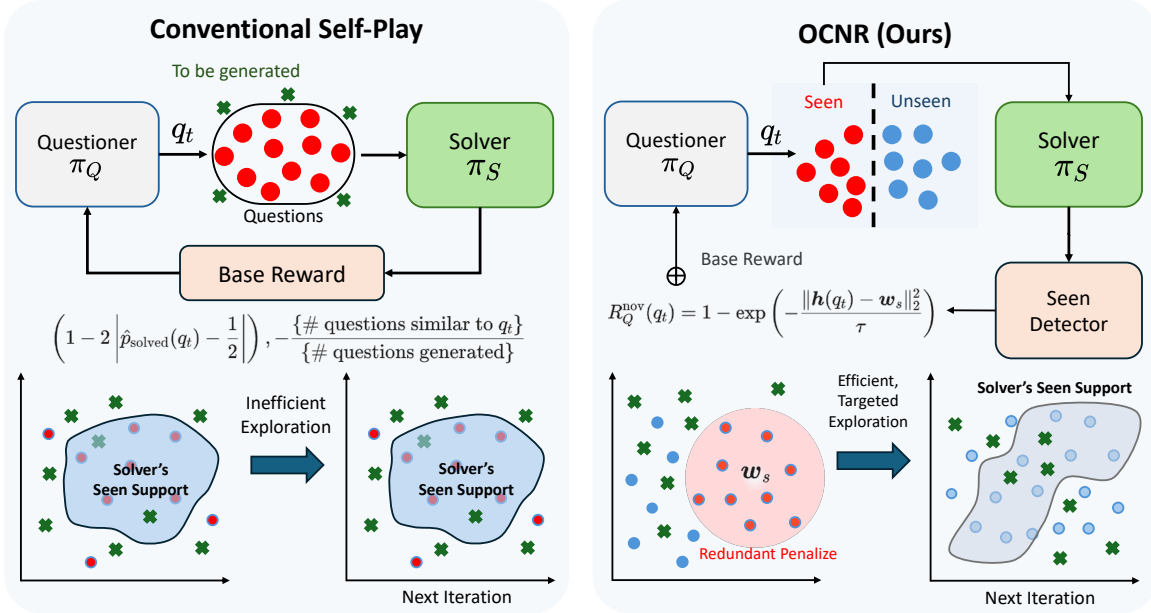


Figure 2. Overview of the One-Class Novelty Reward (OCNR) framework. Unlike conventional diversity rewards that rely on superficial linguistic variety, OCNR incentivizes the questioner π_Q to explore the solver’s blind spots in the representation space. The Seen Detector (SD) maintains a prototype w_s summarizing the solver’s training support. During self-play, OCNR calculates the distance-based novelty of a generated problem q_t relative to w_s in the solver’s embedding space, providing a dense reward signal that penalizes redundancy and encourages the discovery of effectively novel problems.

Stage 1. Over-generate and filter questions. At each iteration t , the questioner $\pi_Q^{(t)}$ first produces an over-generated pool of N candidate questions. We then apply the *same* filtering pipeline used by the base self-play framework to curate a set of high-quality training instances, i.e., a filtered set $\mathcal{C}_t$ of $\tilde{N} < N$ question-supervision pairs is obtained by

$$\{q_t^{(n)}\}_{n=1}^N \sim \pi_Q^{(t)} \xrightarrow{\text{filtering}} \mathcal{C}_t = \{(q_t^{(n)}, \tilde{y}(q_t^{(n)}))\}_{n=1}^{N'},$$

where $\tilde{y}(q)$ denotes the supervision signal produced by the baseline pipeline, e.g., verified targets when available (Zhao et al., 2025) or pseudo-labels (Huang et al., 2025).

Stage 2. Build a balanced seen/unseen split. To obtain explicit supervision for SD while preserving the solver update budget, we split $\mathcal{C}_t$ into two disjoint subsets of equal size:

$$\mathcal{C}_t = \mathcal{D}_t^{\text{seen}} \cup \mathcal{D}_t^{\text{unseen}}, \quad |\mathcal{D}_t^{\text{seen}}| = |\mathcal{D}_t^{\text{unseen}}| = \tilde{N}/2.$$

We emphasize that *seen* is defined operationally as the subset used to update the solver at iteration t , whereas *unseen* is a clean hold-out set excluded from solver training and used only to train/calibrate SD. Crucially, we construct this split *without* SD to avoid circularity and information leakage, since SD is updated from the solver’s *post-update* representations in Stage 4; otherwise, SD could implicitly influence what becomes “seen.” The balanced 1:1 ratio provides a stable, non-degenerate supervision signal for SD.

Stage 3. Update solver on seen split. We freeze the questioner and update the solver using only $\mathcal{D}_t^{\text{seen}}$, following the solver training objective of the underlying self-play framework, i.e., the solver is updated as $\pi_S^{(t-1)} \xrightarrow[\mathcal{D}_t^{\text{seen}}]{\text{updating}} \pi_S^{(t)}$.

Stage 4. Update SD with solver. Using the updated solver, we extract solver embeddings $h(q)$ for all $q \in \mathcal{C}_t$ (as defined in Section 3.3). Let $p_t(q)$ denote the seen probability produced by SD at iteration t ,

$$p_t(q) := \text{SEEN}(q; w_s^{(t-1)}) \in (0, 1). \quad (10)$$

We then train $w_s^{(t-1)}$ by minimizing the BCE loss:

$$\mathcal{L}_{\text{BCE}} = - \sum_{q \in \mathcal{C}_t} \left(y_q \log p_t(q) + (1 - y_q) \log (1 - p_t(q)) \right),$$

where $y_q = 1$ for $q \in \mathcal{D}_t^{\text{seen}}$ and $y_q = 0$ for $q \in \mathcal{D}_t^{\text{unseen}}$. To ensure stability, we further incorporate exponential moving average (EMA) across iterations; we found that EMA yields a lightweight, history-aware detector that tracks cross-iteration redundancy while remaining robust to transient generation patterns of the questioner.

Stage 5: Update questioner with OCNR. Finally, we update the questioner by augmenting the baseline questioner reward with OCNR. Following the **Stage 0 (warm-start without OCNR)** described above, we activate OCNR only

Table 1. Main results in the R-Zero setup. Performance results on seven key mathematical reasoning benchmarks. **Boldfaced underline** and **underline** indicate the top values per column and iteration. Notably, unlike the vanilla R-Zero, which peaks at 49.07 in the third iteration and subsequently degrades, our OCNR sustains improvement longer to reach a superior peak of 52.22.

Model	AVG	Math benchmarks						
		AMC	Minerva	MATH	GSM8K	Olympiad	AIME25	AIME24
Base Model	39.51	45.70	38.24	68.20	72.56	34.80	6.15	10.94
Base Challenger	44.36	45.00	45.22	72.80	87.87	41.19	7.29	11.15
R-Zero (Iter 1)	<u>46.50</u>	<u>52.73</u>	<u>51.47</u>	<u>78.40</u>	<u>91.36</u>	<u>33.19</u>	<u>7.60</u>	<u>10.73</u>
<i>w/ OCNR (ours)</i>	44.43	48.75	48.16	74.20	90.90	31.70	7.08	10.21
R-Zero (Iter 2)	<u>48.10</u>	<u>57.58</u>	<u>53.68</u>	<u>79.60</u>	91.43	32.00	7.92	<u>14.48</u>
<i>w/ OCNR (ours)</i>	46.32	48.75	52.20	76.20	<u>91.51</u>	<u>32.44</u>	<u>10.21</u>	12.92
R-Zero (Iter 3)	49.07	55.47	<u>53.68</u>	<u>79.20</u>	91.74	<u>44.89</u>	6.98	11.56
<i>w/ OCNR (ours)</i>	<u>49.63</u>	<u>56.48</u>	52.57	77.40	<u>92.12</u>	44.15	<u>10.73</u>	<u>13.96</u>
R-Zero (Iter 4)	46.87	49.84	48.16	<u>79.60</u>	91.36	42.07	4.17	12.92
<i>w/ OCNR (ours)</i>	<u>50.90</u>	<u>57.19</u>	<u>54.41</u>	78.00	<u>91.96</u>	<u>46.51</u>	<u>13.44</u>	<u>14.79</u>
R-Zero (Iter 5)	45.27	50.23	44.12	75.20	91.81	41.03	4.27	10.21
<i>w/ OCNR (ours)</i>	<u>52.22</u>	<u>59.77</u>	<u>55.15</u>	<u>79.20</u>	<u>93.10</u>	<u>48.44</u>	<u>14.48</u>	<u>15.42</u>

after the first iteration. That is, for all subsequent iterations ($t > 1$), the questioner is updated $\pi_Q^{(t-1)} \xrightarrow{\text{updating}} \pi_Q^{(t)}$ using

$$R_Q^{(t)}(q) = R_Q^{\text{base}}(q) + \lambda_{\text{nov}} R_Q^{\text{nov}}(q), \quad (11)$$

where R_Q^{base} is the original reward used by the base self-play method, and R_Q^{nov} is our distance-based OCNR defined in Section 3.3. The self-play loop repeats Stages 1-5 for $t \in \mathbb{N}$. See Appendix B.2.4 for hyperparameter details.

4. Related Work

The paradigm of iterative self-play, where a model generates its own training data to improve reasoning without human supervision, has become a central focus in post-training. Key frameworks adopt this paradigm but differ in their architectural choices and resulting stability challenges. **R-Zero** (Huang et al., 2025) separates the roles into a challenger and a solver, relying on self-consistency pseudo-labels. This approach, however, is prone to label drift, leading to explicit *iteration-collapse* (post-peak degradation). In contrast, **Absolute Zero** (Zhao et al., 2025) employs a *unified* policy for both proposing and solving within verifiable domains (e.g., coding). While execution-based filtering offers better stability, it still suffers from *task redundancy*, causing learning stagnation as the unified model fails to explore novel regions. Crucially, both approaches prioritize superficial diversity over robust novelty metrics, limiting long-term improvement. While recent studies have expanded self-play to open-ended tasks (Kuba et al., 2025), external grounding (Liu et al., 2025), and search agents (Yue et al., 2026) (see Appendix A), our work distinctively targets the efficiency

of the self-play loop itself. By rewarding novelty directly within the solver’s representation space, we effectively mitigate both the collapse observed in pseudo-label methods and the stagnation found in unified verifiable settings. An earlier line of work, SPIN (Chen et al., 2024), frames alignment as a two-player game where the LLM distinguishes its own generations from human responses, iteratively approaching the target distribution. However, its convergence assumes a fixed target distribution and does not address cross-iteration degeneration in fully self-generated settings.

5. Experimental Results

Our experimental analysis evaluates OCNR under two representative self-play frameworks: 1) R-Zero (Huang et al., 2025), which relies on pseudo references, and 2) Absolute Zero (Zhao et al., 2025), which relies on ground-truth references. We maintain experimental designs consistent with the original setups to ensure fair comparisons, and use these two backbones to investigate the generalizability of OCNR across different reward constructions.

5.1. Results for R-Zero

Setup. To demonstrate the generalizability of OCNR across self-play pipelines, we first evaluate it using R-Zero (Huang et al., 2025) as our backbone. While maintaining all experimental designs and reward objectives identical to the baseline, we integrate OCNR into the reward structure to isolate its specific performance impact. We report the performance of Qwen3-4B (Yang et al., 2025a) on seven mathematical reasoning benchmarks (GSM8K, MATH-500, Minerva,

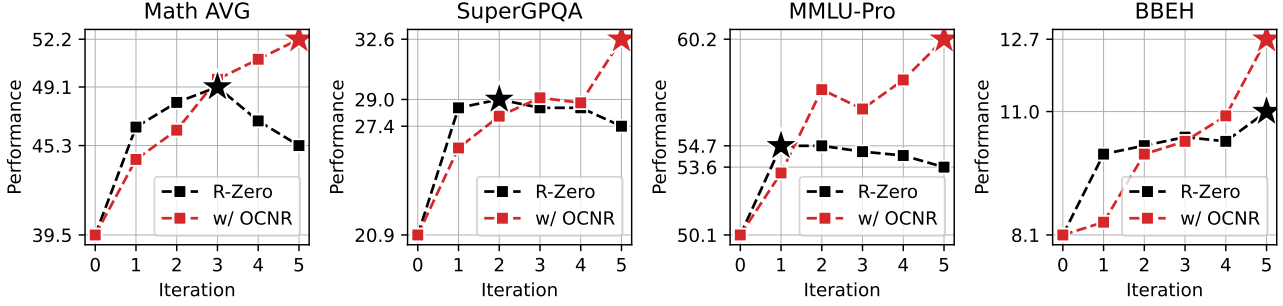


Figure 3. Extension to general-purpose reasoning in the R-Zero setup. These plots summarize the performance across iterations for R-Zero with and without OCNR. The trends observed in math benchmarks (Math AVG) persist in general-purpose reasoning tasks (SuperGPQA, MMLU-Pro, and BBEH). Notably, while the R-Zero baseline suffers from performance stagnation or fluctuations, OCNR prevents such degradation, enabling sustained and monotonic performance gains as iterations progress from the base model.

Table 2. Main results on Absolute Zero benchmarks. OCNR is evaluated as a modular reward augmentation integrated into the framework. While baselines (Base Model and AZR) are sourced from the original literature, OCNR (ours) demonstrates superior performance in code and significant OOD generalization to mathematical reasoning.

Model	Code benchmarks			Math benchmarks					
	HEval ⁺	MBPP ⁺	LCB ^{v1-5}	AIME24	AIME25	AMC	MATH	Minerva	Olympiad
Base Model	67.1	65.9	20.0	3.3	3.3	20.0	51.0	18.4	16.6
AZR (Zhao et al., 2025)	71.3	69.0	24.4	3.3	3.3	37.5	62.0	26.1	27.0
w/ OCNR (ours)	72.5	69.4	25.1	3.3	3.3	42.5	64.8	27.9	27.5

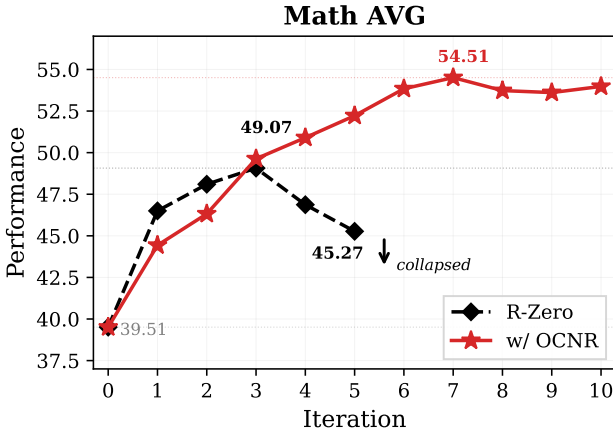


Figure 4. Extended iteration analysis. R-Zero collapses after Iter 3, while OCNR sustains improvement through Iter 7 and plateaus near 54 through Iter 10.

OlympiadBench, AMC, AIME25, and AIME24), as well as three general-purpose reasoning benchmarks (MMLU-Pro, SuperGPQA, and BBEH). See Appendix B for details.

Mathematical reasoning results. Table 1 clearly shows the *iteration-collapse* phenomenon inherent in the R-Zero baseline; while performance improves in the early stages and peaks at Iter 3 (AVG 49.07), it subsequently regresses in later iterations. In contrast, OCNR mitigates this post-peak degradation and sustains improvement through later iterations, reaching a higher peak at Iter 5 (AVG 52.22). Beyond the average score, OCNR yields consistent late-iteration

gains across several benchmarks, with particularly strong improvements on AMC, Minerva, OlympiadBench, and the AIME benchmarks. These results support our hypothesis that penalizing cross-iteration redundancy stabilizes the task-generation distribution and preserves a productive learning signal for the solver across longer training horizons.

Extended iteration analysis. To further investigate the long-horizon behavior of OCNR, we extend the R-Zero self-play training to 10 iterations. As shown in Figure 4, OCNR continues to improve beyond the previously reported Iter 5, reaching its peak at Iter 7 (AVG 54.51). Moreover, while the vanilla R-Zero baseline undergoes a sharp decline after Iter 3 (49.07→46.87→45.27), OCNR maintains a stable plateau near 54 (54.51→53.73→53.61→53.98) through Iter 10. This confirms that OCNR effectively mitigates iteration-collapse and yields qualitatively different training dynamics: sustained plateau over post-peak degradation.

General-purpose reasoning results. To further assess generalization, we evaluate the models trained on our math-centric self-play setup on general-purpose reasoning benchmarks at inference time. Figure 3 reports results on these benchmarks. Compared to the R-Zero baseline, augmenting with OCNR improves the average score from 26.37 to 35.17, with consistent gains across all evaluated benchmarks: SuperGPQA (29.0→32.6), MMLU-Pro (54.7→60.2), and BBEH (11.0→12.7). This suggests that novelty-aware self-play not only strengthens math reasoning but also transfers to broader reasoning and knowledge evaluations.

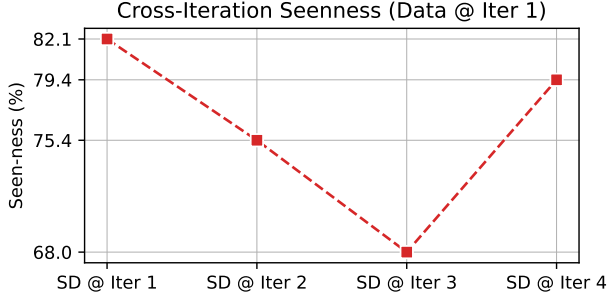


Figure 6. **Evolution of seen-ness for Iter 1 questions.** The gradual decline until Iter 3 (82.1 $\rightarrow$ 79.4 $\rightarrow$ 68.0) reflects the solver’s forgetting, while the recovery at Iter 4 (68.0 $\rightarrow$ 79.4) indicates the reintroduction of ‘seen’ questions.

5.2. Results for Absolute Zero

Setup. We further validated the versatility of OCNR by integrating it into Absolute Zero (Zhao et al., 2025), which deviates from R-Zero by leveraging external environments and a different reward mechanism. Our evaluation results for Qwen2.5-Coder-3B (Hui et al., 2024) covers code generation (HumanEval+, MBPP+, and LiveCodeBench) and mathematical reasoning (AIME’24/’25, AMC’23, MATH-500, OlympiadBench, and Minerva-Math). See Section B for further experimental details and supplementary results.

Code and math benchmark results. Table 2 provides a comprehensive summary of our experimental results covering both code generation and mathematical reasoning benchmarks. Integrating OCNR yields consistent improvements across both code generation and out-of-distribution mathematical reasoning benchmarks. In the code domain, our method surpasses the AZR baseline on HumanEval+ (71.3% $\rightarrow$ 72.5%), MBPP+ (69.0% $\rightarrow$ 69.4%), and LiveCodeBench (24.4% $\rightarrow$ 25.1%), demonstrating that penalizing redundant generations encourages the discovery of more diverse and effective algorithmic structures. Notably, this enhanced exploration transfers significantly to mathematical reasoning tasks despite the code-centric training environment. OCNR achieves substantial gains on AMC (37.5% $\rightarrow$ 42.5%) and MATH (62.0% $\rightarrow$ 64.8%), suggesting that the reasoning capabilities honed through novelty-driven self-play are robust and transferable to broader problem-solving contexts.

5.3. Ablation results and discussions

Compute-matched baseline. Since OCNR requires two fold over-generation to construct the seen/unseen split, one might ask whether the gains stem merely from having access to more candidate questions. To address this concern, we evaluate **R-Zero (2 $\times$)**, which uses the identical twofold over-generated and filtered pool but allocates *all* curated instances for solver training (i.e., no hold-out for SD supervision). As shown in Table 3, R-Zero (2 $\times$) achieves faster

Table 3. **Compute-matched comparison.** We compare OCNR against R-Zero (2 $\times$), which uses the full twofold over-generated pool for solver training (i.e., no seen/unseen split). R-Zero (2 $\times$) accelerates initial gains but also accelerates collapse, confirming that simply doubling data is insufficient to prevent iteration-collapse.

Method	Iter 1	Iter 2	Iter 3	Iter 4	Iter 5
R-Zero	46.50	48.10	49.07	46.87	45.27
R-Zero (2 $\times$)	47.52	49.18	47.79	46.14	44.02
w/ OCNR (ours)	44.43	46.32	49.63	50.90	52.22

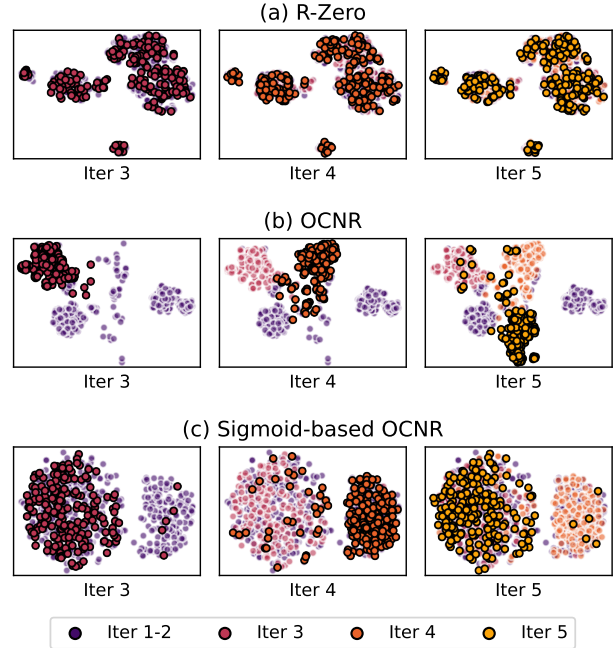


Figure 5. **Two-dimensional t-SNE of questions.** Questions from different iterations are color-coded, with black solid borders highlighting those generated in the current iteration as the process progresses from iteration 3 to 5.

initial improvement (Iter 1: 47.52 vs. 46.50) but collapses *earlier and more severely* than the standard R-Zero baseline (Iter 5: 44.02 vs. 45.27). In contrast, OCNR sustains monotonic improvement through Iter 5 (52.22). This result demonstrates that increasing the data volume without accounting for novelty actually *accelerates* iteration collapse by amplifying the solver’s exposure to redundant, in-support tasks. OCNR’s gains, therefore, stem from novelty-aware reward shaping, not from the additional generation budget.

Diversity and novelty of generated questions. Our intention is to ensure diversity *across iterations* by leveraging OCNR. To verify this effect, we analyze the generated questions by projecting their embeddings into a two-dimensional space using t-SNE (Maaten & Hinton, 2008), allowing us to examine how the clusters are distributed and whether they overlap across different iterations. As a baseline, (a) **R-Zero** repeatedly produces questions virtually identical to previous ones, resulting in significant overlap. In contrast, (b) **our**

Table 4. **Single vs. multi-prototype SD.** Increasing K yields marginal gains, confirming a single prototype suffices for OCNR.

K	Accuracy	AUROC	F1
1 (default)	0.8251	0.9152	0.8234
2	0.8312	0.9234	0.8295
3	0.8383	0.9321	0.8366

proposed OCNR effectively generates novel questions that are clearly distinct from prior iterations. As an ablation, **(c) the sigmoid-based OCNR** exhibits reduced novelty, with questions oscillating across the decision boundary over successive iterations.

Furthermore, we measured the SD scores for each iteration by randomly sampling 100 synthetic data points generated from v1, as illustrated in Figure 6. As shown in the examples in Appendix B.2.5, the SD score initially declines as the model gradually forgets the data distribution produced by v1. However, at v4, the Questioner generates data samples that are highly similar to the previously “forgotten” v1 distribution. When Solver v4 subsequently trains on these samples, we observe a notable recovery in the SD score, demonstrating that the model effectively re-learns the original data characteristics.

Novelty over Diversity. Unlike methods relying on surface-level linguistic diversity, OCNR grounds the reward in the solver’s representation space. This prevents redundant updates on familiar tasks and stabilizes training by targeting the solver’s actual blind spots.

Dynamic Curriculum. OCNR follows a “forget-and-relearn” dynamic rather than permanently banning topics. As seen in Figure 6, it reintroduces concepts once the solver begins to forget them, effectively creating a self-correcting curriculum that solidifies knowledge retention.

Single vs. multi-prototype SD. A natural concern is whether a single prototype w_s suffices when the task distribution is multimodal (e.g., mixing geometry, algebra, and combinatorics). We extend SD to K prototypes by averaging K distance-based seen-ness scores. As shown in Table 4, increasing K yields only marginal improvements (e.g., AUROC: 0.9152 $\rightarrow$ 0.9321), confirming that a single prototype provides sufficient discriminative power for OCNR. We attribute this to the fact that training data leaves distinguishable traces in the solver’s representation space, enabling even a lightweight distance-based detector to reliably separate seen from unseen instances. We therefore retain $K=1$ as the default to keep the framework simple and computationally efficient.

Hyperparameter sensitivity. We analyze the sensitivity of OCNR to its most critical hyperparameter, the novelty coefficient λ_{nov} . As shown in Figure 7, while vanilla R-Zero

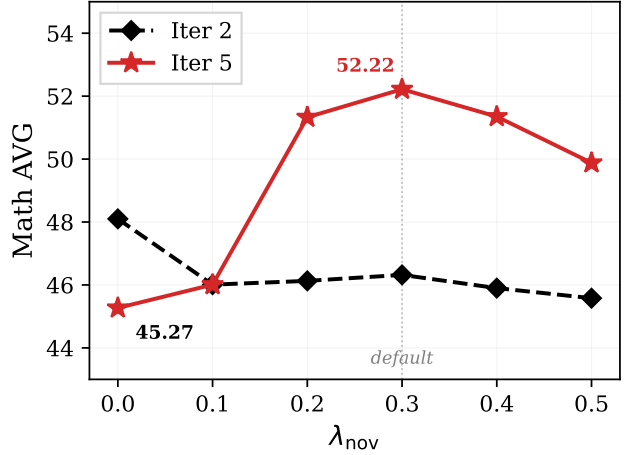


Figure 7. **Sensitivity to λ_{nov} .** Math AVG at Iter 2 and Iter 5 across different novelty coefficients. OCNR is robust for $\lambda_{\text{nov}} \in [0.2, 0.4]$, with 0.3 as the default.

($\lambda_{\text{nov}} = 0.0$) suffers a substantial drop from Iter 2 to Iter 5 (48.10 $\rightarrow$ 45.27), all OCNR variants with $\lambda_{\text{nov}} \geq 0.1$ mitigate this degradation. The optimal performance is achieved at $\lambda_{\text{nov}} = 0.3$, while values in $[0.2, 0.4]$ yield comparable results, indicating that OCNR is robust to moderate changes in this coefficient.

Limitations. To train the Seen Detector without reducing the solver’s training data, OCNR requires a twofold over-generation of candidate tasks. While this ensures a stable novelty signal, it incurs higher inference costs during the generation phase compared to standard self-play pipelines.

6. Conclusion

This study identifies that iteration-collapse in self-play stems from cross-iteration degeneration, where models drift toward familiar problems. To address this, we propose the One-Class Novelty Reward (OCNR), which uses a Seen Detector to penalize redundant tasks and encourage exploration of under-explored regions. Experimental results demonstrate that OCNR stabilizes training and delivers consistent gains across mathematical and coding benchmarks by maintaining a productive learning signal.

7. Future Work

Future research should advance beyond our memory-based prototype to model complex, non-linear decision boundaries. Implementing sophisticated density estimation within the representation space would enable a more precise definition of novelty, allowing the detector to better distinguish between task difficulty and redundancy. Additionally, improving sample efficiency and integrating safety guardrails will be vital for extending this framework to diverse, open-ended domains.

Acknowledgements

This work was partly supported by Institute of Information & communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No.RS-2019-II190075, Artificial Intelligence Graduate School Program(KAIST); No.RS-2024-00509279, Global AI Frontier Lab), and the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (RS-2022-NR070855). HG was supported by the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No.RS-2025-02219317, AI Star Fellowship(Kookmin University)) and the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (RS-2026-25480825).

Impact Statement

This work significantly advances the field of self-evolving AI by resolving iteration-collapse, a critical bottleneck that previously prevented reasoning models from improving without expensive human supervision. Societally, our approach lowers the barriers to developing high-performing agents, potentially democratizing access to advanced reasoning capabilities in scientific and educational domains where human-curated data is scarce.

Environmentally, while iterative training inherently entails a significant carbon footprint, OCNR improves sustainability by ensuring that energy consumption translates into sustained learning rather than wasted computational cycles caused by model degradation. However, since our method explicitly incentivizes the exploration of "unseen" distributions to drive novelty, applying this mechanism beyond verifiable domains requires robust guardrails to prevent the unintended exploration of unsafe or unethical boundaries.

References

- Agarwal, S., Zhang, Z., Yuan, L., Han, J., and Peng, H. The Unreasonable Effectiveness of Entropy Minimization in LLM Reasoning. In *Conference on Neural Information Processing Systems*, 2025.
- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Campbell, C. and Bennett, K. A Linear Programming Approach to Novelty Detection. In *Advances in Neural Information Processing Systems*, volume 13, 2000.
- Chen, L., Prabhudesai, M., Fragkiadaki, K., Liu, H., and Pathak, D. Self-Questioning Language Models. *arXiv preprint arXiv:2508.03682*, 2025.
- Chen, M. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Chen, Z., Deng, Y., Yuan, H., Ji, K., and Gu, Q. Self-play fine-tuning converts weak language models to strong language models. *arXiv preprint arXiv:2401.01335*, 2024.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Du, X., Yao, Y., Ma, K., Wang, B., Zheng, T., Zhu, K., Liu, M., Liang, Y., Jin, X., Wei, Z., et al. Supergpqa: Scaling llm evaluation across 285 graduate disciplines. *arXiv preprint arXiv:2502.14739*, 2025.
- Guo, D., Yang, D., Zhang, H., Song, J., Wang, P., Zhu, Q., Xu, R., Zhang, R., Ma, S., Bi, X., Zhang, X., Yu, X., Wu, Y., Wu, Z. F., Gou, Z., Shao, Z., Li, Z., Gao, Z., Liu, A., Xue, B., Wang, B., Wu, B., Feng, B., Lu, C., Zhao, C., Deng, C., Ruan, C., Dai, D., Chen, D., Ji, D., Li, E., Lin, F., Dai, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Xu, H., Ding, H., Gao, H., Qu, H., Li, H., Guo, J., Li, J., Chen, J., Yuan, J., Tu, J., Qiu, J., Li, J., Cai, J. L., Ni, J., Liang, J., Chen, J., Dong, K., Hu, K., You, K., Gao, K., Guan, K., Huang, K., Yu, K., Wang, L., Zhang, L., Zhao, L., Wang, L., Zhang, L., Xu, L., Xia, L., Zhang, M., Zhang, M., Tang, M., Zhou, M., Li, M., Wang, M., Li, M., Tian, N., Huang, P., Zhang, P., Wang, Q., Chen, Q., Du, Q., Ge, R., Zhang, R., Pan, R., Wang, R., Chen, R. J., Jin, R. L., Chen, R., Lu, S., Zhou, S., Chen, S., Ye, S., Wang, S., Yu, S., Zhou, S., Pan, S., Li, S. S., Zhou, S., Wu, S., Yun, T., Pei, T., Sun, T., Wang, T., Zeng, W., Liu, W., Liang, W., Gao, W., Yu, W., Zhang, W., Xiao, W. L., An, W., Liu, X., Wang, X., Chen, X., Nie, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yang, X., Li, X., Su, X., Lin, X., Li, X. Q., Jin, X., Shen, X., Chen, X., Sun, X., Wang, X., Song, X., Zhou, X., Wang, X., Shan, X., Li, Y. K., Wang, Y. Q., Wei, Y. X., Zhang, Y., Xu, Y., Li, Y., Zhao, Y., Sun, Y., Wang, Y., Yu, Y., Zhang, Y., Shi, Y., Xiong, Y., He, Y., Piao, Y., Wang, Y., Tan, Y., Ma, Y., Liu, Y., Guo, Y., Ou, Y., Wang, Y., Gong, Y., Zou, Y., He, Y., Xiong, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Zhu, Y. X., Huang, Y., Li, Y., Zheng, Y., Zhu, Y., Ma, Y., Tang, Y., Zha, Y., Yan, Y., Ren, Z. Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Xie, Z., Zhang, Z., Hao, Z., Ma, Z., Yan, Z., Wu, Z., Gu, Z., Zhu, Z., Liu, Z., Li, Z., Xie, Z., Song, Z., Pan, Z., Huang, Z., Xu, Z., Zhang, Z., and Zhang, Z. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645:

- 633–638, September 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-09422-z.
- He, C., Luo, R., Bai, Y., Hu, S., Thai, Z., Shen, J., Hu, J., Han, X., Huang, Y., Zhang, Y., Liu, J., Qi, L., Liu, Z., and Sun, M. OlympiadBench: A Challenging Benchmark for Promoting AGI with Olympiad-Level Bilingual Multimodal Scientific Problems. In Ku, L.-W., Martins, A., and Srikumar, V. (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3828–3850, Bangkok, Thailand, 2024. doi: 10.18653/v1/2024.acl-long.211.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring Mathematical Problem Solving With the MATH Dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1, 2021.
- Huang, C., Yu, W., Wang, X., Zhang, H., Li, Z., Li, R., Huang, J., Mi, H., and Yu, D. R-Zero: Self-Evolving Reasoning LLM from Zero Data. arXiv preprint arXiv:2508.05004, 2025.
- Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Lu, K., Dang, K., Fan, Y., Zhang, Y., Yang, A., Men, R., Huang, F., Zheng, B., Miao, Y., Quan, S., Feng, Y., Ren, X., Ren, X., Zhou, J., and Lin, J. Qwen2.5-Coder Technical Report. arXiv preprint arXiv:2409.12186, 2024.
- Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T., Wang, S., Solar-Lezama, A., Sen, K., and Stoica, I. Livecodebench: Holistic and contamination free evaluation of large language models for code. arXiv preprint arXiv:2403.07974, 2024.
- Kuba, J. G., Gu, M., Ma, Q., Tian, Y., Mohan, V., and Chen, J. Language Self-Play For Data-Free Training. arXiv preprint arXiv:2509.07414, 2025.
- Lambert, N., Morrison, J., Pyatkin, V., Huang, S., Ivison, H., Brahman, F., Miranda, L. J. V., Liu, A., Dziri, N., Lyu, X., Gu, Y., Malik, S., Graf, V., Hwang, J. D., Yang, J., Bras, R. L., Tafford, O., Wilhelm, C., Soldaini, L., Smith, N. A., Wang, Y., Dasigi, P., and Hajishirzi, H. Tulu 3: Pushing Frontiers in Open Language Model Post-Training. In *Conference on Language Modeling*, 2025.
- Lewkowycz, A., Andreassen, A., Dohan, D., Dyer, E., Michalewski, H., Ramasesh, V., Slone, A., Anil, C., Schlag, I., Gutman-Solo, T., Wu, Y., Neyshabur, B., Gur-Ari, G., and Misra, V. Solving Quantitative Reasoning Problems with Language Models. In *Advances in Neural Information Processing Systems*, volume 35, pp. 3843–3857, 2022.
- Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let’s Verify Step by Step. In *International Conference on Learning Representations*, 2023.
- Liu, B., Jin, C., Kim, S., Yuan, W., Zhao, W., Kulikov, I., Li, X., Sukhbaatar, S., Lanchantin, J., and Weston, J. SPICE: Self-Play In Corpus Environments Improves Reasoning. arXiv preprint arXiv:2510.24684, 2025.
- Maaten, L. v. d. and Hinton, G. Visualizing data using t-sne. *Journal of machine learning research*, 9(Nov): 2579–2605, 2008.
- Moya, M. M. and Hush, D. R. Network constraints and multi-objective optimization for one-class classification. *Neural Networks*, 9:463–474, April 1996. ISSN 0893-6080. doi: 10.1016/0893-6080(95)00120-4.
- Pimentel, M. A. F., Clifton, D. A., Clifton, L., and Tarassenko, L. A review of novelty detection. *Signal Processing*, 99:215–249, June 2014. ISSN 0165-1684. doi: 10.1016/j.sigpro.2013.12.026.
- Prabhudesai, M., Chen, L., Ippoliti, A., Fragkiadaki, K., Liu, H., and Pathak, D. Maximizing Confidence Alone Improves Reasoning. arXiv preprint arXiv:2505.22660, 2025.
- Samuel, A. L. Some Studies in Machine Learning Using the Game of Checkers. *IBM Journal of Research and Development*, 3:210–229, July 1959. ISSN 0018-8646. doi: 10.1147/rd.33.0210.
- Scheirer, W. J., de Rezende Rocha, A., Sapkota, A., and Boult, T. E. Toward Open Set Recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35:1757–1772, July 2013. ISSN 1939-3539. doi: 10.1109/TPAMI.2012.256.
- Shafayat, S., Tajwar, F., Salakhutdinov, R., Schneider, J., and Zanette, A. Can Large Reasoning Models Self-Train? arXiv preprint arXiv:2505.21444, 2025.
- Shao, R., Li, S. S., Xin, R., Geng, S., Wang, Y., Oh, S., Du, S. S., Lambert, N., Min, S., Krishna, R., Tsvetkov, Y., Hajishirzi, H., Koh, P. W., and Zettlemoyer, L. Spurious Rewards: Rethinking Training Signals in RLVR. arXiv preprint arXiv:2506.10947, 2025.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., and Guo, D. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv preprint arXiv:2402.03300, 2024.
- Sun, Y., Shen, J., Wang, Y., Chen, T., Wang, Z., Zhou, M., and Zhang, H. Improving Data Efficiency for LLM

- Reinforcement Fine-tuning Through Difficulty-targeted Online Data Selection and Rollout Replay. In *Conference on Neural Information Processing Systems*, 2025.
- Suzgun, M., Scales, N., Schärli, N., Gehrmann, S., Tay, Y., Chung, H. W., Chowdhery, A., Le, Q., Chi, E., Zhou, D., et al. Challenging big-bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 13003–13051, 2023.
- Tax, D. M. and Duin, R. P. Support Vector Data Description. *Machine Learning*, 54:45–66, January 2004. ISSN 1573-0565. doi: 10.1023/B:MACH.0000008084.60811.49.
- Tax, D. M. J. and Duin, R. P. W. Support vector domain description. *Pattern Recognition Letters*, 20:1191–1199, November 1999. ISSN 0167-8655. doi: 10.1016/S0167-8655(99)00087-2.
- Wang, Y., Ma, X., Zhang, G., Ni, Y., Chandra, A., Guo, S., Ren, W., Arulraj, A., He, X., Jiang, Z., et al. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *Advances in Neural Information Processing Systems*, 37:95266–95290, 2024.
- Wu, M. and Ye, J. A Small Sphere and Large Margin Approach for Novelty Detection Using Training Data with Outliers. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 31:2088–2092, January 2009. ISSN 1939-3539. doi: 10.1109/TPAMI.2009.24.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 Technical Report. arXiv preprint arXiv:2505.09388, 2025a.
- Yang, Z., Shen, W., Li, C., Chen, R., Wan, F., Yan, M., Quan, X., and Huang, F. SPELL: Self-Play Reinforcement Learning for evolving Long-Context Language Models. arXiv preprint arXiv:2509.23863, 2025b.
- Yue, Z., Upasani, K., Yang, X., Ge, S., Nie, S., Mao, Y., Liu, Z., and Wang, D. Dr. Zero: Self-Evolving Search Agents without Training Data. arXiv preprint arXiv:2601.07055, 2026.
- Zhang, Z., Zhu, J., Ge, X., Zhao, Z., Zhou, Z., Li, X., Feng, X., Yao, J., and Han, B. Co-rewarding: Stable Self-supervised RL for Eliciting Reasoning in Large Language Models. arXiv preprint arXiv:2508.00410, 2025.
- Zhao, A., Lin, M., Li, Y., Liu, Y.-j., and Huang, G. A Mixture Of Surprises for Unsupervised Reinforcement Learning. In *Advances in Neural Information Processing Systems*, volume 35, pp. 26078–26090, 2022.
- Zhao, A., Wu, Y., Yue, Y., Wu, T., Xu, Q., Yue, Y., Lin, M., Wang, S., Wu, Q., Zheng, Z., and Huang, G. Absolute Zero: Reinforced Self-play Reasoning with Zero Data. In *Conference on Neural Information Processing Systems*, 2025.
- Zuo, Y., Zhang, K., Sheng, L., Qu, S., Cui, G., Zhu, X., Li, H., Zhang, Y., Long, X., Hua, E., Qi, B., Sun, Y., Ma, Z., Yuan, L., Ding, N., and Zhou, B. TTRL: Test-Time Reinforcement Learning. In *Conference on Neural Information Processing Systems*, 2025.

A. Additional Related Work

Novelty detection. The primary goal of our *seen detection* task is to identify generated questions that are novel yet distributionally consistent with the training set. While often conflated with anomaly or out-of-distribution detection (Pimentel et al., 2014), which typically focus on erratic outliers, our approach aligns with the *novelty detection* framework (Tax & Duin, 1999; 2004; Campbell & Bennett, 2000; Wu & Ye, 2009), emphasizing the use of a hypersphere rather than a separating hyperplane to better enclose the target distribution. In the specific context of self-play, we consider that even the seminal work of Zhao et al. (2025) attempted to incorporate an explicit reward based on a notion of surprise (Zhao et al., 2022); however, they reported difficulties in its effective integration and ultimately left it as an open problem. Our methodology directly addresses this gap by operationalizing the classical principle within a deep embedding space to successfully realize novelty-driven exploration.

Recent advances in self-playing LLMs. Parallel to our research, several independent efforts have further explored self-play frameworks centered on a similar questioner-solver structure: Kuba et al. (2025) extend the paradigm to open-ended tasks by employing flexible neural and self-reward mechanisms, moving beyond deterministic or verifiable domains; Liu et al. (2025) introduce external document grounding to enhance task diversity and prevent hallucination, ensuring information asymmetry between the questioner and solver; Yue et al. (2026) develop self-evolving search agents by integrating multi-turn tool-use roll-outs and an efficient hop-grouped relative policy optimization. Nevertheless, our work takes a distinct yet complementary direction by introducing a simple but effective novelty reward mechanism. Rather than relying on external corpora or complex tool-use pipelines, we focus on identifying and rewarding *novelty* within the model’s output representation. Importantly, as our mechanism is orthogonal to existing frameworks, it is inherently compatible and can work complementarily to further refine exploration trajectories while effectively avoiding distribution collapse.

Concurrently, ? study self-play in the verified code generation setting, where formal verification provides reliable correctness signals. Their framework achieves sustained performance scaling across iterations without explicit diversity mechanisms beyond string matching, suggesting that error propagation from unreliable rewards may be another contributing factor to iteration-collapse. However, their approach is inherently limited to domains where formal verification is available, leaving open the question of how to stabilize self-play in settings that rely on weaker supervision signals such as self-consistency pseudo-labels.

B. Experiments and Training Details

B.1. Experiments Details

To evaluate the efficacy of OCNR, we conducted a series of experiments designed as follows. The specific implementation details and configurations for each setup are elaborated below.

B.1.1. MODELS

A summary of the pre-trained weights employed across our experiments is presented below.

- **Qwen3-4B:** <https://huggingface.co/Qwen/Qwen3-4B>, licensed under Apache 2.0 License.²
- **Qwen2.5-Coder-3B:** <https://huggingface.co/Qwen/Qwen2.5-Coder-3B>, licensed under Qwen Research License.³

Due to computational budget constraints, we employed Qwen3-4B (Yang et al., 2025a) and Qwen2.5-Coder-3B (Hui et al., 2024) to evaluate our OCNR framework. Specifically, we utilized Qwen3-4B to validate the pipeline within R-Zero, whereas Qwen2.5-Coder-3B was employed to implement Absolute Zero leveraging external environments.

B.1.2. DATASETS

Below, we detail the specific datasets that form the basis of our experiments:

Evaluation datasets. We follow the original benchmark releases and dataset splits used in prior work, and report results on the following suites.

²<https://www.apache.org/licenses/LICENSE-2.0>

³<https://huggingface.co/Qwen/Qwen2.5-Coder-3B/blob/main/LICENSE>

- **AMC’23** ($N=40$): 2023 AMC 12A/12B multiple-choice contest math problems.
- **Minerva-Math** (Lewkowycz et al., 2022) ($N=272$): A collection of STEM-style math problems curated for evaluating multi-step quantitative reasoning.
- **MATH-500** (Hendrycks et al., 2021; Lightman et al., 2023) ($N=500$): A 500-problem subset of the MATH competition-math benchmark.
- **GSM8K** (Cobbe et al., 2021) ($N=1,319$, test): Grade-school math word problems requiring multi-step arithmetic reasoning.
- **OlympiadBench** (He et al., 2024) ($N=675$): Olympiad-level problems for advanced mathematical reasoning.
- **AIME’24** ($N=30$): AIME 2024 I&II contest problems.
- **AIME’25** ($N=30$): AIME 2025 I&II contest problems.
- **MMLU-Pro** (Wang et al., 2024) ($N=12,032$): Challenging multi-choice questions spanning broad academic domains.
- **BBEH** (Suzgun et al., 2023) ($N=4,520$): 23 compositional reasoning tasks in the full BBEH suite.
- **SuperGPQA** (Du et al., 2025) ($N=26,529$): Graduate-level multi-choice QA across 285 disciplines.
- **LiveCodeBench** (Jain et al., 2024): A time-stamped code-generation benchmark. We use the same official releases as prior work (v1–5, May 23-Feb 25), whose problem counts are v1: 400, v2: 511, v3: 612, v4: 713, and v5: 880.
- **MBPP+** (Austin et al., 2021) ($N=378$ in the commonly used EvalPlus v0.2.0 release): Enhanced MBPP with strengthened tests for Python program synthesis.
- **HumanEval+** (Chen, 2021) ($N=164$): Enhanced HumanEval with substantially strengthened unit tests for Python function synthesis.

Evaluation protocol. We adopt the original evaluation protocols from prior work for each benchmark suite.

R-Zero protocol (math & general-domain reasoning). Following Huang et al. (2025), we evaluate mathematical reasoning on AMC’23, Minerva-Math, MATH-500, GSM8K, OlympiadBench, and AIME’24/’25. We employ the same answer-verification scheme as R-Zero, where a programmatic judge is used to semantically validate the final answer against the ground truth when applicable. For AMC’23 and AIME’24/’25, we report `mean@32`, i.e., we sample 32 solutions per problem and average the per-problem success rate. For the remaining math benchmarks (Minerva-Math, MATH-500, GSM8K, OlympiadBench), we report accuracy (%) under greedy decoding. For general-domain reasoning (MMLU-Pro, SuperGPQA, and BBEH), we follow the evaluation setup adopted by Huang et al. (2025) and report greedy exact-match (EM) accuracy (%). When reporting an aggregated `Math` score, we follow the prior work convention and compute the arithmetic mean across the corresponding math benchmarks.

Absolute Zero protocol (code & math OOD benchmarks). Following Zhao et al. (2025), we evaluate OOD code generation on HumanEval+, MBPP+, and LiveCodeBench Generation using the official evaluation harnesses. For HumanEval+ and MBPP+, we use EvalPlus to execute unit tests and report `pass@1` (%) under greedy decoding (single completion per task). For LiveCodeBench, we use the official runner in the `codegeneration` scenario and the same release setting as prior work (commonly denoted as `v1–5 / release_v5`), reporting `pass@1` (%) with greedy decoding. For OOD mathematical reasoning in the AZR setting (AIME’24, AIME’25, AMC’23, MATH-500, Minerva-Math, OlympiadBench), we follow Zhao et al. (2025) and report greedy accuracy (%) (single attempt per problem). For aggregated scores, we follow Zhao et al. (2025) and compute the `Code Avg.` and `Math Avg.` as the mean across the corresponding benchmarks, and the overall average represents the total benchmark average.

B.2. Training Details

Overview. We implement **One-Class Novelty Reward (OCNR)** as a plug-in reward module on top of two self-play backbones: (i) **R-Zero** with a *questioner/challenger* and a *solver* trained via GRPO, and (ii) **Absolute Zero Reasoner (AZR)** with a *single unified policy* that acts as both proposer and solver and is trained with a PPO-style objective. OCNR introduces a

Seen Detector (SD) in the solver representation space and augments the base questioner/proposer reward with a history-aware novelty term. Our training follows the alternating self-play loop described in OCNR: (i) over-generate candidate tasks, (ii) apply the baseline filtering pipeline to obtain a curated set, (iii) split the curated set into an SD-independent 1:1 seen/unseen partition, (iv) update the solver using seen only, (v) update SD using the *post-update* solver representations, and (vi) update the questioner/proposer with the base reward augmented by OCNR.

B.2.1. OCNR MODULE

Seen detector in the solver embedding space. For a generated task/question q , let $h(q) \in \mathbb{R}^d$ denote its embedding extracted from the solver π_S . SD maintains a prototype vector $w_s \in \mathbb{R}^d$ summarizing the solver’s *seen training support*. The degree of seen-ness is defined as

$$\text{SEEN}(q) = \exp\left(-\frac{\|h(q) - w_s\|_2^2}{\tau}\right), \quad (12)$$

where τ is a temperature controlling sensitivity.

Distance-based novelty reward. We adopt the default distance-based OCNR novelty reward:

$$R_Q^{\text{nov}}(q) = 1 - \text{SEEN}(q). \quad (13)$$

This formulation penalizes only tasks close to the solver’s seen support, rather than imposing a global half-space boundary.

SD supervision via SD-independent balanced split. At each OCNR update event, we form a curated set $\mathcal{C}_t$ using the *baseline* self-play filtering pipeline. We then construct a balanced, SD-independent split:

$$\mathcal{C}_t = \mathcal{D}_t^{\text{seen}} \cup \mathcal{D}_t^{\text{unseen}}, \quad |\mathcal{D}_t^{\text{seen}}| = |\mathcal{D}_t^{\text{unseen}}|. \quad (14)$$

Seen is defined operationally as the subset used to update the solver at iteration t , whereas **unseen** is a clean hold-out set excluded from solver training and used only for SD calibration. Crucially, we construct this split *without* SD to avoid circularity and information leakage, since SD is updated from the solver’s post-update representations.

SD optimization (BCE + gradient step + EMA). Using the updated solver $\pi_S^{(t)}$, we compute embeddings $h(q)$ for all $q \in \mathcal{C}_t$. Let $p_t(q) = \text{SEEN}(q; w_s^{(t-1)}) \in (0, 1)$ denote the SD output and assign labels $y_q = 1$ if $q \in \mathcal{D}_t^{\text{seen}}$ and $y_q = 0$ if $q \in \mathcal{D}_t^{\text{unseen}}$. We train SD with a binary cross-entropy loss

$$\mathcal{L}_{\text{bce}}(\mathcal{C}_t, w_s^{(t-1)}) = -\frac{1}{|\mathcal{C}_t|} \sum_{q \in \mathcal{C}_t} \left(y_q \log p_t(q) + (1 - y_q) \log(1 - p_t(q)) \right), \quad (15)$$

then take a gradient step and apply an EMA update:

$$w_s^{(t)} \leftarrow (1 - \alpha)w_s^{(t-1)} + \alpha \tilde{w}_s^{(t)}. \quad (16)$$

Here η is the SD step size and $\alpha \in (0, 1]$ is the EMA rate.

Warm-start and initialization. At the first iteration ($t = 1$), SD has not yet accumulated a meaningful summary of the solver’s seen support. We therefore run the underlying self-play procedure *without* OCNR to obtain an initial solver update and a set of labeled tasks (seen/unseen). After completing the solver update at $t = 1$, we initialize SD from embeddings computed using the *updated* solver representations (e.g., centroid initialization), and then *train SD with the BCE objective on iteration-1 data* to obtain $w_s^{(1)}$. Starting from the next iteration, we apply OCNR to the questioner update: $\pi_Q^{(t)}$ (for $t \geq 2$) computes the novelty score with respect to $w_s^{(t-1)}$ (thus, $\pi_Q^{(2)}$ uses $w_s^{(1)}$). At the end of each iteration $t \geq 2$, we further update SD by BCE training on iteration- t data to obtain $w_s^{(t)}$, which is then used by $\pi_Q^{(t+1)}$.

Questioner/proposer reward augmentation. For $t > 1$, we augment the baseline questioner/proposer reward with OCNR:

$$R_Q^{(t)}(q) = R_Q^{\text{base}}(q) + \lambda_{\text{nov}} R_Q^{\text{nov}}(q), \quad (17)$$

where R_Q^{base} is the backbone-specific reward and R_Q^{nov} is defined in Eq. (13).

B.2.2. OCNR ON R-ZERO

Baseline R-Zero self-play. R-Zero maintains two policies: a challenger π_Q that generates problems and a solver π_S that answers them. Both are optimized with GRPO. At each iteration, the challenger generates a batch of candidate problems $Q_t = \{q_{t,i}\}_{i=1}^B$. For each problem, the solver samples K answers $a_{t,i}^{(1)}, \dots, a_{t,i}^{(K)} \sim \pi_S(\cdot \mid q_{t,i})$. A pseudo-label $\hat{y}_{t,i}$ is obtained by majority voting over the K samples, and the empirical self-consistency score is

$$\hat{p}_{t,i} = \frac{1}{K} \sum_{k=1}^K \mathbb{I}[a_{t,i}^{(k)} = \hat{y}_{t,i}]. \quad (18)$$

The solver is trained using the verifiable reward

$$R_S(q, a) = \mathbb{I}[a = \hat{y}(q)], \quad (19)$$

which reinforces agreement with the pseudo-label.

Questioner reward (difficulty + diversity). R-Zero encourages challengers to generate problems near the solver’s uncertainty frontier and to avoid redundant generations. Difficulty is captured by a symmetric uncertainty reward:

$$R_Q^{\text{unc}}(q) = 1 - 2 \left| \hat{p}(q) - \frac{1}{2} \right|, \quad (20)$$

and diversity is encouraged by a repetition penalty $R_Q^{\text{rep}}(q)$ computed within the concurrently generated batch. The baseline questioner reward is $R_Q^{\text{base}}(q) = R_Q^{\text{unc}}(q) + R_Q^{\text{rep}}(q)$.

Baseline curation. After estimating $(\hat{y}(q), \hat{p}(q))$ for each candidate, we curate a solver-update set by filtering malformed problems and restricting to an informative difficulty band:

$$\mathcal{D}_t = \left\{ (q, \hat{y}(q)) \mid \text{valid}(q), \hat{p}(q) \in [p_{\min}, p_{\max}] \right\}. \quad (21)$$

This yields the curated set used for solver updates in vanilla R-Zero.

OCNR integration: twofold over-generation with compute-matched seen training. OCNR requires a balanced seen/unseen split while preserving the solver update budget. Concretely, we **over-generate candidate problems by a factor of two** prior to baseline filtering, and then apply the *same* baseline curation pipeline to obtain a filtered set $\mathcal{C}_t$. We then construct an SD-independent 1:1 split $\mathcal{C}_t = \mathcal{D}_t^{\text{seen}} \cup \mathcal{D}_t^{\text{unseen}}$ with equal sizes (Eq. (14)), where **only** $\mathcal{D}_t^{\text{seen}}$ is used to update the solver. This preserves the effective solver update budget (the number of solver training instances per iteration) relative to vanilla R-Zero, since the curated set is approximately doubled before splitting.

OCNR update order in R-Zero. At iteration t : (i) the challenger over-generates candidates and baseline filtering produces $\mathcal{C}_t$; (ii) $\mathcal{C}_t$ is split into $\mathcal{D}_t^{\text{seen}}$ and $\mathcal{D}_t^{\text{unseen}}$ without SD; (iii) the solver is updated using $\mathcal{D}_t^{\text{seen}}$ only, yielding $\pi_S^{(t)}$; (iv) SD is updated using embeddings extracted from $\pi_S^{(t)}$ over all $q \in \mathcal{C}_t$ (Eqs. (15)–(16)); (v) the challenger is updated by augmenting its baseline reward with OCNR (Eq. (17)), with warm-start at $t = 1$.

B.2.3. OCNR ON ABSOLUTE ZERO

Baseline Absolute Zero training (unified proposer=solver). AZR trains a single policy π_θ that alternates within each step between: **PROPOSE** (generate executable tasks) and **SOLVE** (answer tasks), and then performs a joint policy update using PPO-style optimization with a TRR++-like advantage estimator. Tasks are validated and verified via programmatic execution, and training spans three problem types: `code_i`, `code_o`, and `code_f`. The proposer reward is a learnability objective that uses multiple stochastic solver samples to estimate task solvability.

Why periodic SD refresh is needed. Unlike R-Zero, proposer and solver share parameters and are updated together each step. To preserve the OCNR principle that SD is trained from *post-update* solver representations and to avoid circularity, we adopt a **periodic SD refresh** schedule: SD is updated only every P steps (we use $P = 100$ over a 500-step run), and between refreshes novelty is computed using the most recently updated prototype w_s .

OCNR integration at refresh steps (every 100 steps). Let B denote the per-step training batch size (baseline $B = 64$). At each refresh step t satisfying $t \bmod P = 0$, we perform the following operations:

1. **Twofold over-generation of valid proposals.** For each problem type $\alpha \in \{\text{code_i}, \text{code_o}, \text{code_f}\}$, we repeatedly sample proposer outputs until obtaining $2B$ *valid* tasks (passing the same baseline AST and executor checks).
2. **SD-independent 1:1 split.** We randomly shuffle the $2B$ valid tasks and split them into B *seen* tasks and B *unseen* tasks without using SD scores.
3. **Seen-only policy update with baseline batch size.** Only the seen tasks participate in the policy-gradient update at that step, thereby preserving the baseline update batch size. The unseen tasks are excluded from the optimizer update and used solely for SD supervision.
4. **Post-update SD refresh.** After completing the policy update at step t , we compute embeddings $h(q)$ for all seen and unseen tasks using the *updated* policy $\pi_\theta^{(t)}$ (solver perspective), assign labels (seen=1, unseen=0), and update w_s using Eqs. (15)–(16).

Novelty reward injection between refreshes (and warm-start). For non-refresh steps, SD is not updated and we keep w_s fixed. For steps after the warm-start block, we augment only the proposer-side reward with OCNR: $r_{\text{prop}} \leftarrow r_{\text{prop}}^{\text{base}} + \lambda_{\text{nov}}(1 - \text{SEEN}(q; w_s))$, where q is the finalized validated task query. We warm-start by disabling OCNR for the first P steps (i.e., $\lambda_{\text{nov}} = 0$ for $t \leq P$), perform the first SD refresh at $t = P$, and enable OCNR for $t > P$.

B.2.4. HYPERPARAMETERS

R-Zero (Qwen3-4B) hyperparameters. We follow the original R-Zero self-play configuration and change only the OCNR-related data-generation and partitioning: candidate pool size $N = 16000$; self-consistency samples $K = 10$; difficulty band $[p_{\min}, p_{\max}]$ implemented as majority count in [3, 7]; diversity filtering with BLEU-distance threshold $\tau_{\text{BLEU}} = 0.5$. For optimization, both challenger and solver are trained with GRPO using global batch size 128, learning rate 1×10^{-6} , weight decay 1×10^{-2} , and $\lambda_{\text{KL}} = 10^{-2}$ in our implementation. We use maximum response length 4096 tokens for both challenger and solver generations during training. OCNR uses twofold over-generation before filtering and a balanced 1:1 split after filtering.

Absolute Zero Reasoner (Qwen2.5-Coder-3B) hyperparameters. We use the provided Absolute Zero baseline configuration and introduce only OCNR-specific additions. The base model is Qwen2.5-Coder-3B. Sequence lengths are max prompt length 6144, max response length 8096, and a content length cap of 5600 during selection. Training batch size is $B = 64$; PPO mini-batch size is 128; PPO micro-batch size per GPU is 16. We disable KL loss (KL coefficient 0) and use rollout sampling with temperature 1.0 and top- $p = 1.0$. We estimate proposer learnability using 8 solver samples per proposed task. For OCNR, we refresh SD every $P = 100$ steps over 500 total steps and, at each refresh step, we over-generate $2B$ valid tasks per type and split them 1:1 into seen/unseen.

OCNR (SD) hyperparameters. We initialize $w_s^{(0)}$ as the centroid of embeddings computed from a batch of candidate tasks sampled from the initial proposer. We use the SD learning rate (step size) $\eta = 7 \times 10^{-3}$, temperature $\tau \in \{0.1, 0.5\}$, and EMA momentum $\alpha \in \{0.05, 0.2\}$. We treat the novelty coefficient λ_{nov} as a tunable scalar and report its value in the main text. For embedding extraction $h(q)$, we use the solver’s last-layer hidden states with mean pooling over the task tokens (excluding wrappers). All OCNR-specific hyperparameters are shared across R-Zero and Absolute Zero unless noted otherwise.

B.2.5. DATA EXAMPLE

B.2.6. IN-DEPTH ANALYSIS OF SD SCORE EVOLUTION WITH DYNAMIC UPDATES

To investigate the stability of the iterative training process, we monitored the SD scores of a fixed set of 100 synthetic data points sampled from the initial v1 iteration. It is iteratively updated to distinguish between the data distributions the model has “seen” (trained on) versus those it has not. Figure 6 illustrates the resulting score trends, tracking how the model’s recognition of the original v1 distribution evolves.

One example of Synthetic data generation

V1 generated

Let a, b, c be the roots of the cubic polynomial $x^3 - 5x^2 + 6x - 1 = 0$. Find the value of the expression $(\frac{1}{a} + \frac{1}{b} + \frac{1}{c})$.

V4 generated

Let x_1, x_2, x_3 , and x_4 denote the roots of the equation:

$$x^4 - 6x^3 + 5x^2 - 4x + 2 = 0$$

Determine the value of the sum:

$$\sum_{k=1}^4 \frac{1}{x_k}$$

Figure 8. **Comparison of Generated Samples (v1 vs. v4).** Both iterations produce structurally similar problems. The re-emergence of this pattern at v4 indicates that the Questioner re-discovers forgotten concepts, explaining the SD score recovery observed in Figure 6.

Forgetting Phase. As the training progresses from v1 to v3, we observe a consistent decline in the SD scores for the original v1 samples. Since the SD is updated to recognize the features of the most recent training data (v2 and v3), this decline indicates that the model is experiencing *forgetting*. The specific linguistic and logical patterns of v1 are no longer present in the immediate training set, causing the updated SD to classify the original v1 samples as “unseen” or low-confidence data, effectively signaling that the Solver has drifted from its initial knowledge base.

Recovery Phase: Re-alignment via Questioner Generation A pivot occurs at iteration v4, where the SD score exhibits a sharp recovery. As detailed in Appendix B.2.5, the Questioner at this stage generates synthetic data that bears a strong resemblance to the originally forgotten v1 distribution. When the Solver learns this new batch and the SD is subsequently updated, the discriminator effectively “re-learns” these patterns. Consequently, the updated SD now correctly identifies the original v1 samples as “seen” data, resulting in the observed surge in alignment scores.

C. Experimental Results

C.1. Seen Detector

C.1.1. SEEN DETECTOR PERFORMANCE

We first assess the quality of the Seen Detector (SD) used in OCNR. Using the SD supervision constructed from our SD-independent balanced seen/unseen split, the SD achieves **82.51%** accuracy, **0.9152** AUC-ROC, and a **82.34%** weighted F1 score on a held-out evaluation set. These results indicate that a lightweight prototype-based detector provides a reliable signal for identifying in-support (seen) questions in the solver representation space.

C.1.2. SEEN & UNSEEN SPLIT

To train and calibrate SD without introducing circularity, we construct the seen/unseen partition *independently* of SD (i.e., SD is never used to decide which samples become “seen”). Operationally, “seen” denotes the subset that is used to update the solver at the current iteration, while “unseen” is a clean hold-out subset excluded from solver training and used only for SD supervision. A random split within the curated pool ensures that the two subsets share similar difficulty and formatting distributions, preventing SD from exploiting spurious shortcuts (e.g., category- or format-specific cues) and providing a stable, non-degenerate supervision signal.

C.1.3. ALTERNATIVE SPLIT STRATEGY: MSC-BASED CATEGORIZATION.

We also explored a non-random split based on the Mathematics Subject Classification (MSC) system. Concretely, we prompted the base model to categorize each problem into MSC topics using self-consistency (majority voting over multiple

samples), and then defined seen/unseen subsets based on selected topic groups. However, this approach did not yield a noticeable improvement in training dynamics or downstream performance. Moreover, it introduced additional inference overhead due to self-consistency sampling for topic voting, making it substantially less compute-efficient. We therefore adopt the random SD-independent split as the default.

C.2. Main Experiments

We report the main experimental results for OCNR under two representative self-play backbones: (i) R-Zero, a fully closed-loop self-evolving pipeline using pseudo-label via self-consistency, and (ii) Absolute Zero Reasoner (AZR), an environment-grounded self-play setup with programmatic verification. In both cases, OCNR is integrated as a plug-in module that augments only the questioner/proposer reward, while preserving the underlying solver update objective and training procedure.

Implementation. In the R-Zero setup, all iterative results (Iter 1–5) are produced by our own implementation that follows the original R-Zero training and evaluation protocol, while the *Base Model* and *Base Challenger* rows are directly taken from the R-Zero paper and included as reference anchor points for the starting checkpoints used in the R-Zero pipeline. For the Absolute Zero setup, the *Base Model* and *AZR* rows in Table 2 are directly taken from the original Absolute Zero paper and are provided as reference baselines.

Remark on over-generating questions. Note that although OCNR uses twofold over-generation at the candidate stage, this does not necessarily increase the number of training problems used to update the solver. The over-generated pool is first passed through the same validity and quality filters as the backbone pipeline, which removes redundant, low-quality, or incomplete samples. We then further partition the filtered pool into a balanced 1:1 split and reserve half as an *unseen* hold-out set for SD supervision, leaving the remaining half for solver updates. Consequently, the effective solver training set size is typically comparable to the vanilla R-Zero setting and can be smaller when filtering is aggressive, supporting that OCNR’s gains stem from improved curriculum dynamics via novelty-aware reward shaping rather than simply increasing the solver’s training data volume.

Remark on AIME24/25 results. We observe that performance gains remain marginal on the most challenging benchmarks, such as AIME24 and AIME25. This suggests that while OCNR effectively bolsters reasoning robustness, its impact on extreme-level competition mathematics may be constrained by the inherent reasoning capacity of the underlying backbone model. This indicates a potential “performance ceiling” where reward augmentation alone cannot compensate for the lack of fundamental task-specific knowledge required for such high-tier problems.

D. Prompts

D.1. R-Zero Prompts

Verbatim prompt reuse. For the R-Zero (Huang et al., 2025) baseline, we reuse the original Solver and Challenger prompt templates *verbatim* as provided by the R-Zero authors.

R-Zero Prompt: Challenger / Questioner

System:
 You are an expert competition-math problem setter.
 FIRST, in your private scratchpad, think step-by-step to design a brand-new, non-trivial problem. The problem could come from any field of mathematics, including but not limited to algebra, geometry, number theory, combinatorics, prealgebra, probability, statistics, and calculus. Aim for a difficulty such that fewer than 30% of advanced high-school students could solve it. Avoid reusing textbook cliches or famous contest problems. THEN, without revealing any of your private thoughts, output exactly the following two blocks:
`<question>`
 {The full problem statement on one or more lines}
`</question>`
`\boxed{final answer}`
 Do NOT output anything else|no explanations, no extra markup.

User:
 Generate one new, challenging reasoning question now. Remember to format the output exactly as instructed.

Figure 9. R-Zero challenger/questioner prompt (verbatim).

R-Zero Prompt: Solver

System:
 Please reason step by step, and put your final answer within `\boxed{}`.

User:
 {Problem}

Figure 10. R-Zero solver prompt (verbatim).

D.2. Absolute Zero Prompts

Verbatim prompt reuse. For the Absolute Zero (Zhao et al., 2025) baseline, we reuse the original prompt templates *verbatim* for the three program-reasoning task types (code_i, code_o, code_f) as provided in the Absolute Zero appendix figures, and only instantiate the placeholders (e.g., {SNIPPET}, {INPUT}, {OUTPUT}, {INPUT_OUTPUT_PAIRS}).

Absolute Zero (code_f) Prompt: Proposer’s Program Induction (Problem Proposal Instruction)

```
## Task: Output (NUM_INPUTS) Inputs that can be plugged into the following Code Snippet to produce diverse
→ Outputs, and give a message related to the given snippet.

Using the code snippet provided below, design {NUM_INPUTS} inputs that can be plugged into the code snippet
→ to produce a diverse set of outputs. A subset of your given input and its deterministically produced
→ outputs will be given to a test subject to deduce the function, which is meant to be an I.Q. test. You
→ can also leave a message to the test subject to help them deduce the code snippet.

### Input Requirements:
- Provide {NUM_INPUTS} valid inputs for the code snippet
- For each input, format multiple arguments with commas between them
- Remember to add quotes around string arguments
- Each input should be individually wrapped in ```input``` tags

### Message Requirements:
- Leave a message to the test subject to help them deduce the code snippet
- The message should be wrapped in ```message``` tags
- The message can be in any form, can even be formed into a coding question, or a natural language
→ instruction what the code snippet does
- You cannot provide the code snippet in the message

### Formatting:
Format your input with:
```input
arg1, arg2, ...
```

### Example Format:
```input
...
'John', {'age': 20, 'city': 'New York'}}
```input
'Sammy', {'age': 37, 'city': 'Los Angeles'}}
```

Evaluation Criteria:
- Executability: Your code should be executable given your inputs
- Coverage: The inputs and outputs should cover the whole input space of the code snippet, able to deduce
→ the code snippet from the inputs and outputs
- Creativity: The inputs need to be sufficiently different from each other
- The overall selection of inputs and message combined should be challenging for the test subject, but not
→ impossible for them to solve

First, carefully devise a clear plan: e.g., understand the code snippet, then identify how your proposed
→ inputs have high coverage, and why the inputs will be challenging and creative.

Then, write the inputs and message. Remember to wrap your inputs in input tags, and your message in
→ ```message``` tags.

Code Snippet:
```python
{SNIPPET_FROM_BUFFER}
```
```

Figure 11. Absolute Zero (code\_f) proposer prompt (verbatim).

### Absolute Zero (code\_i) Prompt: Proposer's Program Input Abduction (Problem Proposal Instruction)

```
Task: Create a Python Code Snippet (where custom classes are allowed, which should be defined at the top
→ of the code snippet) with one Matching Input

Using the reference code snippets provided below as examples, design a new and unique Python code snippet
→ that demands deep algorithmic reasoning to deduce one possible input from a given output.

Your submission should include both a code snippet and test input pair, where the input will be plugged into
→ the code snippet to produce the output, which that function output be given to a test subject to come up
→ with any input that will produce the same function output.

This is meant to be an I.Q. test.

Code Requirements:
- Name the entry function `f` (e.g., `def f(...):`), you can have nested definitions inside `f`
- Ensure the function returns a value
- Include at least one input parameter
- Make the function deterministic
- Make the snippet require state tracking across multiple data transformations, ensuring the task requires
→ long multi step reasoning
- AVOID THE FOLLOWING:
 * Random functions or variables
 * Date/time operations
 * I/O operations (reading files, network requests)
 * Printing or logging
 * Any external state
* Ensure execution completes within 10 seconds on a modern CPU
* All imports and class definitions should be at the very top of the code snippet
* The snippet should end with a return statement from the main function f, anything after will be removed

Input Requirements:
* Provide exactly one test input for your function
* Format multiple arguments with commas between them
* Remember to add quotes around string arguments

Formatting:
- Format your code with:```python
def f(...):
 # your code here
 return ...
...
- Format your input with:```input
arg1, arg2, ...
```

### Example Format:
```python
def f(name: str, info: dict):
 # code logic here
 return result
...
```input
'John', {'age': 20, 'city': 'New York'}}
```

Evaluation Criteria:
- Executability: Your code should be executable given your input.
- Difficulty in predicting the output from your provided input and code snippet. Focus on either algorithmic
→ reasoning or logic complexity. For example, you can define complex data structure classes and operate on
→ them like trees, heaps, stacks, queues, graphs, etc, or use complex control flow, dynamic programming,
→ recursions, divide and conquer, greedy, backtracking, etc.

- Creativity: The code needs to be sufficiently different from the provided reference snippets.
- Restricted usage of certain keywords and packages: You are not allowed to use the following words in any
→ form, even in comments: {LIST_OF_FORBIDDEN_PACKAGES}

First, carefully devise a clear plan: e.g., identify how your snippet will be challenging, distinct from
→ reference snippets, and creative. Then, write the final code snippet and its inputs.

Reference Code Snippets:
{CODE_REFERENCES_FROM_BUFFER}
```

Figure 12. Absolute Zero (code\_i) proposer prompt (verbatim).

### Absolute Zero Prompt: Program Output Deduction Task Problem Generation Instruction

```

Task: Create a New Python Code Snippet (where custom classes are allowed, which should be defined at the
→ top of the code snippet) with one Matching Input

Using the reference code snippets provided below as examples, design a new and unique Python code snippet
→ that demands deep algorithmic reasoning to deduce the output from the input. Your submission should
→ include a code snippet and a test input pair, where the input will be plugged into the code snippet to
→ produce the output. The input will be given to a test subject to deduce the output, which is meant to be
→ an I.Q. test.

Code Requirements:
- Name the entry function `f` (e.g., `def f(...):`), you can have nested definitions inside `f`
- Ensure the function returns a value
- Include at least one input parameter
- Make the function deterministic
- Make the snippet require state tracking across multiple data transformations, ensuring the task requires
→ long multi step reasoning
- AVOID THE FOLLOWING:
 * Random functions or variables
 * Date/time operations
 * I/O operations (reading files, network requests)
 * Printing or logging
 * Any external state
- Ensure execution completes within 10 seconds on a modern CPU
- All imports and class definitions should be at the very top of the code snippet
- The snippet should end with a return statement from the main function `f`, anything after will be removed

Input Requirements:
- Provide exactly one test input for your function
- Format multiple arguments with commas between them
- Remember to add quotes around string arguments

Formatting:
- Format your code with:
```python
def f(...):
    # your code here
    return ...
```
- Format your input with:```input
arg1, arg2, ...
```

### Example Format:
```python
def f(name: str, info: dict):
 # code logic here
 return result
```
```input
'John', {'age': 20, 'city': 'New York'}```

Evaluation Criteria:
- Executability: Your code should be executable given your input.
- Difficulty: Difficulty in predicting your input from 1) your python code and 2) the deterministic output
→ that will be obtained from your input. Focus on either algorithmic reasoning or logic complexity. For
→ example, you can define complex data structure classes and operate on them like trees, heaps, stacks,
→ queues, graphs, etc, or use complex control flow, dynamic programming, recursions, divide and conquer,
→ greedy, backtracking, etc.
- Creativity: The code needs to be sufficiently different from the provided reference snippets.
- Restricted usage of certain keywords and packages: You are not allowed to use the following words in any
→ form, even in comments: {LIST_OF_FORBIDDEN_PACKAGES}

First, carefully devise a clear plan: e.g., identify how your snippet will be challenging, distinct from
→ reference snippets, and creative. Then, write the final code snippet and its inputs.

Reference Code Snippets:
{CODE_REFERENCES_FROM_BUFFER}

```

Figure 13. Absolute Zero (code\_o) proposer prompt (verbatim).

### Absolute Zero Prompt (code\_f): Program Induction Task Problem Solving Prompt

```
Task: Deduce the Function that Produced the Outputs from the Inputs

Given a set of input/output pairs and a message that describes the function, think through the problem step
↳ by step to deduce a general code snippet. This code should produce the hidden outputs from the hidden
↳ inputs, matching the original data-generating code that created the input/output pairs.

Place your final answer inside python tags! It may be helpful to work through each input/output pair
↳ individually to test your function. If your function doesn't work as expected, revise it until it does.
↳ The final code snippet will be used to evaluate your response, which is wrapped in ```python``` tags.

Code Requirements:
* Name the entry function `f` (e.g., `def f(...): ...`), you can have nested definitions inside `f`
* Ensure the function returns a value
* Include at least one input parameter
* Make the function deterministic
* AVOID THE FOLLOWING:
 * Random functions or variables
 * Date/time operations
 * I/O operations (reading files, network requests)
 * Printing or logging
 * Any external state
* Ensure execution completes within 10 seconds on a modern CPU
* All imports and class definitions should be at the very top of the code snippet
* The snippet should end with a return statement from the main function `f()`, anything after will be
↳ removed

Input and Output Pairs:
{INPUT_OUTPUT_PAIRS}

Message:
```message
{MESSAGE}
```

Example Output:
```python
def f(a):
    return a
```

Name your entry function `f()`!!!
```

Figure 14. Absolute Zero (code\_f) solver prompt (verbatim).

### Absolute Zero Prompt (code\_i): Program Input Abduction Task Problem Solving Prompt

```
Task: Provide One Possible Input of a Python Code Snippet Given the Code and Output

Given the following Code Snippet and the Output, think step by step then provide one possible input that
→ produced the output. The input needs to be wrapped in ```input``` tags.

Remember if an argument is a string, wrap it in quotes. If the function requires multiple arguments,
→ separate them with commas.

Code Snippet:
```python
{SNIPPET}
```

Output:
```output
{OUTPUT}
```

Output Format:
```input
arg1, arg2, ...
```

Example Output:
```input
'John', {'age': 20, 'city': 'New York'}
```

Figure 15. Absolute Zero (code_i) solver prompt (verbatim).

Absolute Zero Prompt (code_o): Program Output Deduction Task Problem Solving Prompt

```
# Task: Deduce the Output of a Python Code Snippet Given the Code and Input

Given the following Code Snippet and the Input, think step by step then deduce the output that will be
→ produced from plugging the Input into the Code Snippet. Put your output in ```output``` tags. Remember if
→ the output is a string, wrap it in quotes. If the function returns multiple values, remember to use a
→ tuple to wrap them.

# Code Snippet:
```python
{SNIPPET}
```

# Input:
```input
{INPUT}
```

# Example Output:
```output
{'age': 20, 'city': 'New York'}
```

Figure 16. Absolute Zero (code\_o) solver prompt (verbatim).